Abstract Class dan Interface

1. Abstract Class
2. Abstract method()
3. Memahami Interface
4. Deklarasi dan syarat Interface
5. Implementasi Interface
6. Interface vs Class
7. Implementasi vs Inheritance
8. Interface vs abstract Class
9. Contoh aplikasi Interface

abstract class

- Suatu Class biasanya dapat dinstansiasi menjadi objek, misal class Dosen dapat diinstansiasi (diwujudkan) menjadi Budi, Heri, dan Heru, namun class MahlukHidup atau class Benda walaupun masih dapat dipaksakan untuk diinstansiasi, namun tidak dapat mewakili keragaman objek di class tersebut sehingga sulit diinstansiasi, class seperti inilah yang disebut abstract class.
- abstract class yang objeknya sangat heterogen ini terlalu umum, sehingga memerlukan class Turunan (subclass) yang lebih homogen dan lebih nyata sehingga dapat diistansiasi/diwujudkan objeknya.

abstract method()

- Bila class MahlukHidup memiliki mehtod(): makan(), minum(), bernafas(), bersuara(), dan reproduksi().
- Sangat sulit dicari persamaan cara bernafas, cara bernafas juga heterogen (dengan paru-paru, insang, stomata, dll), demikian pula bersuara() karena setiap MahlukHidup berbeda-beda/beragam cara bersuara() akan sangat spesifik cara masing-masing species bersuara().
- Method() seperti itulah yang dikatagorikan sebagai abstract method().
- Dibutuhkan class turunan (subclass) yang lebih khusus, lebih nyata, dan lebih homogen untuk melakukan ***override*** terhadap mehtod() yang abstract tersebut.

memahami Interface (1)

- Interface dapat diilustrasikan sebagai **profesi**/**peran**.
- Interface adalah class yang ***paling umum*** atau ***paling abstract*** atau ***paling heterogen*** sehingga ***semua*** method() pada interface harus abstract, artinya semua method() harus diimplementasikan oleh kelompok objek yang masih memiliki kesamaan cara (homogenitas cara) bagaimana method() tersebut dilakukan.
- Dalam kehidupan nyata sehari-hari dapat diketahui ada ***manusia*** yang bekerja sebagai **da'i, dosen, dokter, atlet, tentara, penyanyi, pengacara**, dan sebagainya, manusia-manusia tsb selain memiliki method() yg ***umum*** sebagai seorang manusia, juga harus memiliki method() ***khusus*** yg sesuai dengan profesinya.
- (h

Contoh

- Interface Manusia {
- void mengajar();
- void mendiagnosa();
- void berlatih_perang();
- void menyanyi();
- }
- public class Dokter **implements** Manusia {
- public void mendiagnosa() {
- .....;
- }
- }

memahami Interface (2)

- Sehingga untuk membuat objek seorang Budi berprofesi sebagai seorang Dosen, maka harus dibuat class misal ***DosenIF*** sebagai turunan (subclass) dari class ***Manusia*** namun mengimplementasikan interface ***Dosen***.
- Misal ***interface Hewan*** memiliki abstract method ***bersuara()***, maka masing-masing kelompok hewan sesuai dengan speciesnya akan mengimplementasikan method ***bersuara()*** karena memiliki cara sendiri-sendiri dalam ***bersuara()***.

Contoh

- Interface MahlukHidup {
- makan();
- minum();
- }
- Interface Hewan ~~implements~~ **extends** MahlukHidup {
- bersuara();
- makan();
- bernafas();
- bereproduksi();
- }
- public Class Kucing **implements** Hewan {
- public bersuara() {
- System.out.println(”meong”);
- }
- }

Deklarasi dan ketentuan Interface

- [modifier] **interface** ***NamaInterface*** [extends
- NamaInterface1, NamaInteface2 [, NamaInterfaceN]] {
- [deklarasi konstanta]
- [modifier1] ***tipeReturnValue namaMethod([arguments]);***
- }

- Umumnya nama Interface berupa pasangan **verb+able**, misalnya **Runnable**, **Serializable**, **Calculatable,** atau **noun** yang bermakna **profesi**.

- Ketentuan interface:
- 1. Attributenya hanya berupa konstanta.
- 2. Tidak memiliki objek sehingga tidak ada instansiasi.
- 3. Tidak memiliki modifier static.
- 4. Tidak ada **implements** dari/ke interface lain.
- 5. Tidak boleh **extends** dari/ke class lain.
- 6. Boleh **extends** ke lebih dari satu interface lain.

Implementasi Interface

- Suatu class boleh implements method-method dari berbagai interface (library referensi) tanpa harus menurunkannya.
- Template/bentuk umum implementasi:
- *[modifier] class NamaClass implements NamaInterface1,*
- *[,NamaInterfaceN] {*
- //pengunaan konstanta
- **//implementasi method()**
- *}*

Interface vs Class

Komponen	Interface	Class
Definisi	Daftar deklarasi konstanta dan daftar abtract method()	Model objek
Kandungan informasi	Implementasi/isi semua method() nya berada di luar interface	Mendefiniskan attribute dan mathod() secara konkret dan rinci
Instansiasi	Tidak boleh	Boleh

Implementasi vs Inheritance

- Inheritance adalah proses pewarisan attribute atau pewarisan method() atau constructor() dari suatu super class ke satu atau lebih sub class.
- Bagaimana kalau dibutuhkan suatu class yang attribute nya dan method() nya berasal dari dua super class atau lebih? Disinilah keterbatasan inheritance.
- Suatu interface boleh memasukkan method-method dari berbagai library referensi tanpa harus menurunkannya.
- Nama-nama interface boleh dijadikan tipe data attribute ataupun tipe parameter di dalam class yang menggunakan interface tersebut.
- Interface dan abstract class sama-sama tidak boleh ada intansiasi objek.

Interface vs Abstract Class

Komponen	Interface	Abstract class
attribute	Hanya berupa konstanta	Bebas
method()	Hanya berupa deklarasi	Sebagian boleh berupa deklarasi abstract method(). Sebagian boleh berupa method() lengkap.
Syntax	Seluruhnya abstract, Semua method() wajib dioverride oleh subclass yg mengimplementasikannya	Sebagian boleh abstract, Semua method() yang abstract wajib dioverride oleh subclass nya.

Contoh aplikasi Interface

- Interface seringkali digunakan untuk menghandle event pada aplikasi GUI, perhatikan contoh project ***pFauna*** dan project ***pAppJButton.***